

Nama : Muhammad Defrizal

NPM : 23312111

Kelas : IF 23 D

Mata Kuliah : PBS

Tahap 2 – Implementasi JWT Authentication dan Proteksi Endpoint

1. Pendahuluan

Pada tahap kedua pengembangan Digi-Mount Rajabasa, dilakukan implementasi sistem autentikasi menggunakan JSON Web Token (JWT) pada API Gateway serta penerapan mekanisme proteksi endpoint untuk meningkatkan keamanan komunikasi antar layanan.

Implementasi ini bertujuan agar hanya pengguna yang telah terautentikasi yang dapat mengakses endpoint tertentu serta memastikan bahwa service backend hanya dapat diakses melalui API Gateway.

2. JWT Authentication

2.1 Tujuan

JWT Authentication digunakan untuk:

- Memverifikasi identitas pengguna.
- Menghasilkan Access Token dan Refresh Token.
- Mengamankan endpoint yang membutuhkan autentikasi.
- Menjadi dasar sistem otorisasi pada tahap pengembangan berikutnya.

2.2 Implementasi

Fitur autentikasi dikembangkan pada API Gateway menggunakan:

- NestJS
- Passport
- Passport-JWT
- @nestjs/jwt

Struktur modul autentikasi:

```
auth/
├── dto/
├── guards/
├── strategies/
├── auth.controller.ts
├── auth.module.ts
└── auth.service.ts
```

3. Login Endpoint

Endpoint login digunakan untuk melakukan autentikasi pengguna.

Endpoint

POST /api/auth/login

Request Body

```
{
  "email": "admin@gmail.com",
  "password": "admin123"
}
```

Response

```
{
  "success": true,
  "message": "Access Token Berhasil Dibuat",
  "data": {
    "access_token": "...",
    "refresh_token": "..."
  }
}
```

Pada tahap ini data pengguna masih menggunakan static credential untuk kebutuhan pembelajaran JWT.

4. Access Token dan Refresh Token

Access Token

Digunakan untuk mengakses endpoint yang membutuhkan autentikasi.

Karakteristik:

- Secret: access-gunung-rajabasa-oke
- Masa berlaku: 15 menit

Refresh Token

Digunakan untuk memperoleh Access Token baru ketika Access Token telah kedaluwarsa.

Karakteristik:

- Secret: refresh-gunung-rajabasa-oke
 - Masa berlaku: 7 hari
-

5. JWT Strategy

JWT Strategy digunakan untuk melakukan validasi token yang dikirim oleh client.

Access Strategy

Menggunakan Bearer Token yang dikirim melalui header:

Authorization: Bearer <token>

Refresh Strategy

Menggunakan refresh token yang dikirim melalui request body:

```
{  
  "refresh_token": "..."  
}
```

6. JWT Guard

JWT Guard digunakan untuk melindungi endpoint yang membutuhkan autentikasi.

Guard akan:

- Mengecek keberadaan token.
- Memvalidasi token menggunakan strategy yang sesuai.
- Menolak akses apabila token tidak valid.

Komponen yang digunakan:

- JwtAccessGuard
 - JwtRefreshGuard
-

7. Refresh Token Endpoint

Endpoint refresh digunakan untuk membuat Access Token baru menggunakan Refresh Token yang masih valid.

Endpoint

POST /api/auth/refresh

Request Body

```
{  
  "refresh_token": "..."  
}
```

Response

```
{  
  "success": true,  
  "message": "Access Token Berhasil Dibuat",  
  "data": {  
    "access_token": "..."  
  }  
}
```

8. Proteksi Endpoint Jalur

Endpoint Jalur pada API Gateway telah dilindungi menggunakan JWT Access Guard.

Contoh penggunaan:

```
@UseGuards(JwtAccessGuard)
```

Dengan demikian pengguna wajib mengirimkan Access Token yang valid untuk mengakses endpoint Jalur.

9. Internal Guard

Selain JWT Authentication, diterapkan juga mekanisme Internal Guard pada Service Jalur.

Tujuan:

- Mencegah akses langsung ke Service Jalur.
- Memastikan request hanya berasal dari API Gateway.

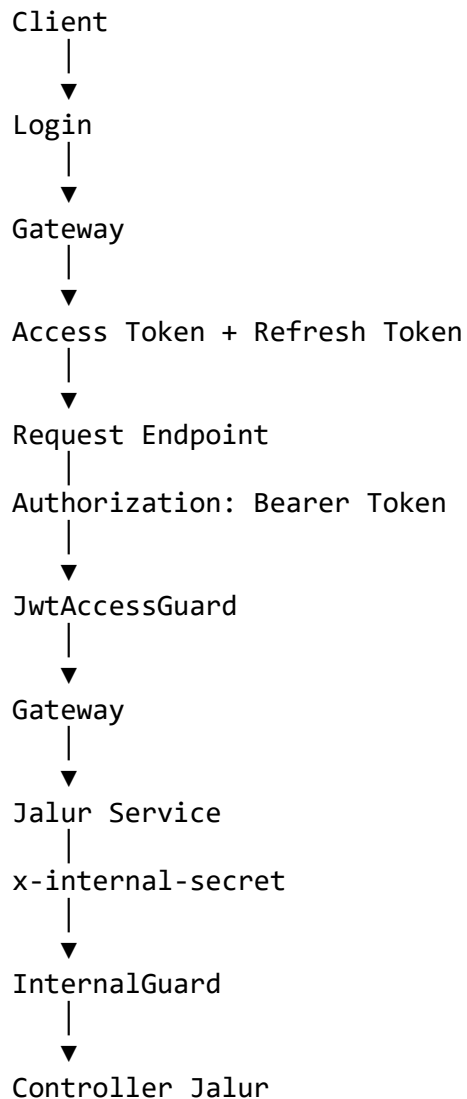
Service Jalur akan memeriksa header:

```
x-internal-secret
```

Jika nilai secret tidak sesuai maka request akan ditolak dengan status:

401 Unauthorized

10. Alur Autentikasi



11. Hasil Implementasi

Tahap kedua berhasil mengimplementasikan:

- JWT Authentication.
- Access Token.
- Refresh Token.

- JWT Strategy.
- JWT Guard.
- Endpoint Refresh Token.
- Proteksi Endpoint Jalur.
- Internal Guard antar service.
- Komunikasi aman antara Gateway dan Jalur Service.

Implementasi ini menjadi fondasi keamanan sebelum melanjutkan pengembangan service berikutnya seperti Pendaftaran, Logistik, dan Payment.